StoryMaker - Run wild with your ideas!

Author: Vikram Bhatt **Version:** 1.0.0

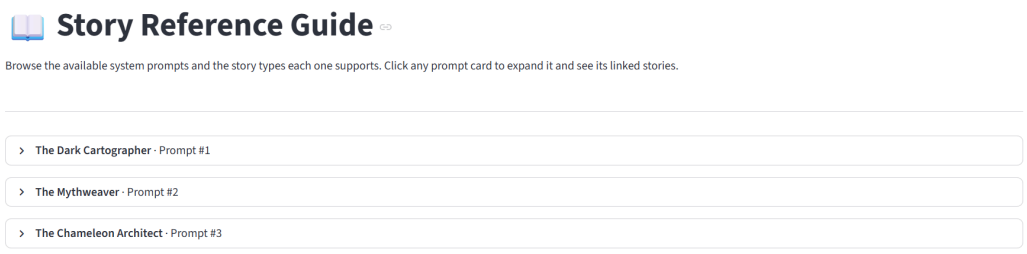
Description

The StoryMaker project is about using AI to build out short stories of different types ranging from fantasy to sci-fi to even more! I use OpenRouter to query the Arcee AI free model that is excellent at creative writing, story-telling, and much more! Additionally, the project is Object Oriented and uses classes to build out the story. It is a work in progress and there are many things I still want to add.

Features

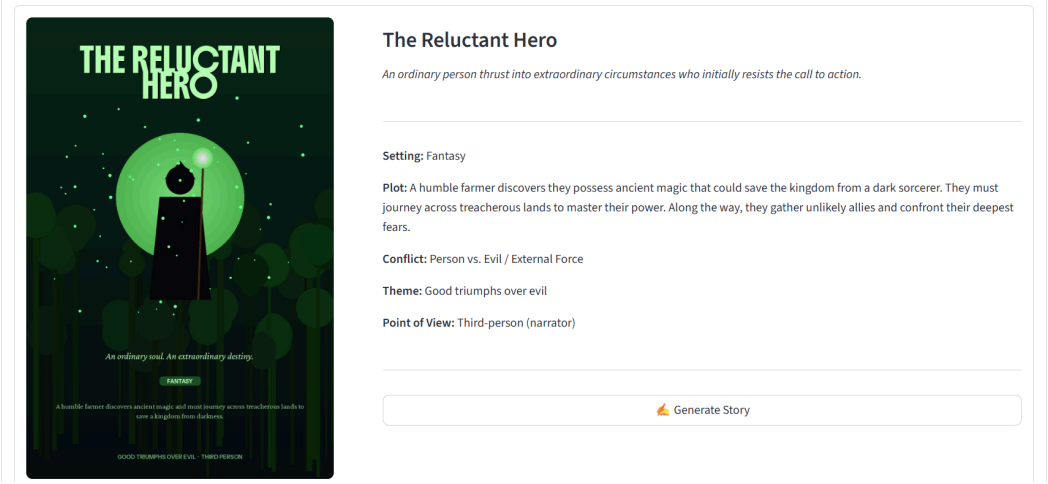
Browse writing styles

Choose from multiple AI writing styles — each one shapes the tone, voice, and structure of the generated story differently. Styles are displayed as collapsible cards so the page stays clean until you need them.



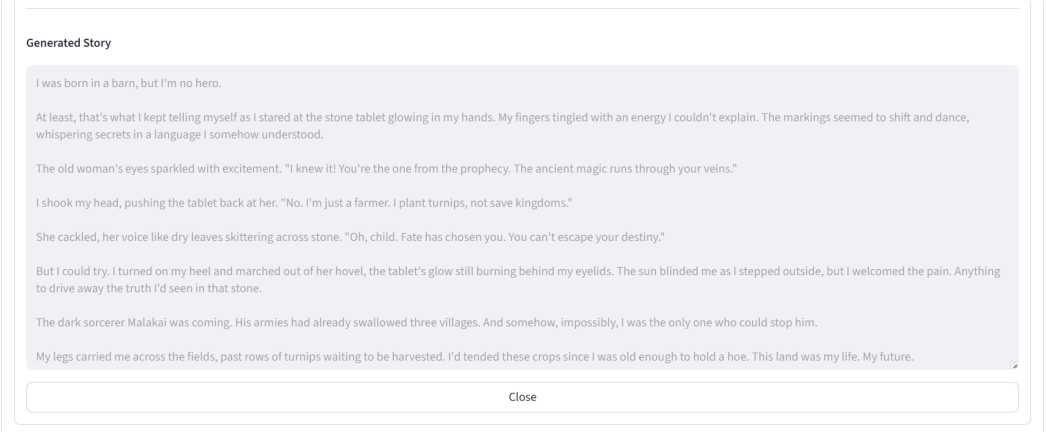
Explore story archetypes with poster art

Each writing style is linked to a set of story archetypes. Expand any style to see its stories — every card shows a generated poster image alongside the full story brief: protagonist, setting, plot, conflict, theme, and point of view.



Generate and read stories inline

Hit **Generate Story** to stream an AI-written story directly into the card. The story persists across page interactions and can be dismissed with the **Close** button when you're done.



Getting Started

This project uses uv for dependency and virtual environment management. **Python 3.12 or higher is required.**

1. Install uv

macOS / Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/inst
```

Or via pip (any platform):

```
pip install uv
```

2. Create the virtual environment and install dependencies

```
uv sync
```

This reads `pyproject.toml`, creates a `.venv` folder, and installs all required packages automatically.

3. Set up your API key

Create a `.env` file in the project root and add your OpenRouter API key:

```
OPENROUTER_API=your_api_key_here
```

You can get a free key at openrouter.ai.

4. Run the project

Script (CLI):

```
uv run main.py
uv run main.py --build StoryMaker --history true
```

Streamlit app:

```
uv run streamlit run app.py
```

File Structure

```
project/
│
├── app.py                  # Streamlit web app — browse story types
├── main.py                 # CLI script — generate and update a sto
│
├── StoryMaker.py           # Core class: handles API calls, convers
├── StoryHelper.py          # Extends StoryMaker: loads story data a
│
├── story_inputs/           # Static data used by StoryHelper
│   ├── story_types.json        # 10 story archetypes with character
│   ├── story_system_prompts.json  # System prompts that shape the r
│   ├── posters/                # Poster images for each story arche
│   └── generate_posters.py     # Script used to generate the poster
│
├── outputs/                # Generated at runtime by main.py
│   ├── updated_story.txt       # The updated story output
│   └── first_convo.json        # Full conversation history in JSON
│
├── pyproject.toml          # Project metadata and dependencies (uv)
├── uv.lock                 # Locked dependency versions
├── .env                    # API key — not committed to version con
└── README.md               # This file
```

Dependencies

All dependencies are managed via `pyproject.toml` and installed with `uv sync`.

Library	Version	Purpose
`openai`	>=2.21.0	OpenAI-compatible SDK used to call the OpenRouter API
`python-dotenv`	>=1.2.1	Loads the API key from the `.env` file
`streamlit`	>=1.30	Web app framework for the interactive story browser
`Pillow`	>=10.0.0	Loads and displays story poster images

Next Steps

- [ ] **Implement the `--build StoryHelper` CLI flag** — `main.py` parses the argument but the `StoryHelper` branch is never executed
- [ ] **Add error handling for failed API calls** — if the primary model and all fallbacks are unavailable, the app currently raises an unhandled exception

- [ ] **Improve the Streamlit UI** — add the ability to customise protagonist name, setting, and plot directly in the app before generating
- [ ] **Update the assignment notebook** — `*args`, generators, decorators, and argparse are all implemented in the code but not yet marked as done in the tracking notebook